

SCALEMORPH

Frontera Scheduling API

Backend Developer Onboarding Guide

Training project — NestJS · TypeScript · AWS (Lambda, S3, SES) · Docker ·
Supabase · security · cloud document patterns · Claude Code · critical
reasoning

Repo: `scaleMorph/frontera` · Version 2.2 · June 2026

Contents

1. Welcome — what this project is (and isn't)
2. Your learning goals & cloud focus areas
3. 30-day path on Frontera
4. Module 1 — TypeScript & NestJS on Frontera
5. Module 2 — Cloud: file storage (S3)
6. Module 3 — Cloud: compute (Docker + Serverless)
7. Module 4 — Cloud: messaging & documents (SES, PACR, OCR concepts)
8. Module 5 — Security (JWT, RLS, secrets)
9. Module 6 — Working with Claude Code
10. Module 7 — Critical reasoning (don't mind-dump on AI)
11. Understand → Design → Implement → Verify
12. Week-by-week checklist
13. Reading list & repo map
14. 30-day sign-off

1. Welcome — what this project is (and isn't)

Welcome to **Scalemorph**. Your onboarding project is the **Frontera Scheduling API** — a NestJS backend for healthcare provider scheduling (onboarding, availability, time-off, approvals, calendar exports).

Repo path: `/Users/hamzajamshed/scaleMorph/frontera` (or your team's clone of the same repository).

This is a training & delivery project — not production QuoteLogik. Frontera is where you learn our stack, contribute to a real codebase, and grow into cloud-backed backend work. You will not be dropped onto a live customer system on day one.

What Frontera is

- **NestJS 11 + TypeScript** on Node 22
- **Supabase Postgres** + Drizzle ORM (20+ tables)
- **AWS Serverless** — Lambda + API Gateway (`serverless.yml`)
- **AWS S3** — secure document storage (PACR PDFs, exports)
- **AWS SES** — provider invite and notification email
- **Docker Compose** — local Postgres + migrate + API with hot reload

Day-one setup

```
cd scaleMorph/frontera
npm run docker:up          # Postgres + migrations + seed + API on :3000

# Verify
curl http://localhost:3000/health
open http://localhost:3000/api    # Swagger
```

Alternative: cloud Supabase — see `README.md` Option B.

Phased delivery (Q1–Q4)

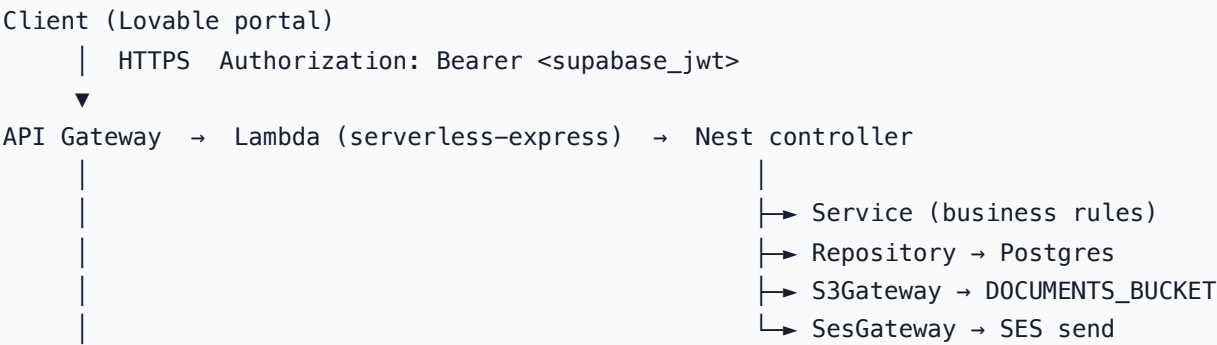
You'll grow with the roadmap. Today's screen-backed routes are documented in `docs/adr/` :

- Admin: active providers, onboard provider, master availability, schedule change approvals, PRN availability
- Provider portal: scheduling submissions (PRN + set time-off)
- Coming: JWT guard (Q1), PACR uploads to S3 (Q2), review queue + SES liaison (Q3), calendar export (Q4)

2. Your learning goals & cloud focus areas

Cloud domain	What you'll learn	Where in Frontera
File storage	S3 buckets, PutObject/GetObject, bucket env vars, gateway pattern	<code>src/repository/aws/s3.gateway.ts</code> , <code>DOCUMENTS_BUCKET</code> , <code>EXPORTS_BUCKET</code>
Compute	Container local dev vs Lambda production; shared <code>createApp()</code> bootstrap	<code>docker-compose.yml</code> , <code>src/lambda.ts</code> , <code>serverless.yml</code>
Messaging / email	SES identities, configuration sets, sending from Lambda IAM role	<code>src/repository/aws/ses.gateway.ts</code> , invite flow docs
Documents & OCR (concepts)	PACR PDF upload pipeline; later: cloud OCR patterns (read → store → link <code>documents</code> row)	<code>provider-documents.service.ts</code> , <code>documents</code> table — OCR is a <i>learning module</i> , not wired yet
Security	JWT verification, RLS helpers, secrets, IDOR awareness	<code>docs/api-auth.md</code> , <code>drizzle/0001_supabase_rls_functions.sql</code>
Data layer	Repository pattern, migrations, seed scripts	<code>src/repository/persistence/</code> , <code>drizzle/</code>

Frontera request path (local or Lambda – same Nest app)



3. 30-day path on Frontera — full curriculum

Goal by day 30: grasp the *entire* onboarding curriculum — NestJS, TypeScript, S3, SES, Docker, Serverless, security (JWT/RLS/secrets), document & OCR concepts, Claude Code workflow, and critical reasoning — then prove it via a merged PR and the sign-off checklist in section 14. Every module in this document maps to a week below; nothing is “for later.”

3.1 What you will grasp (all within 30 days)

Topic	Module	When	Proof you got it
TypeScript & NestJS (DI, modules, repositories)	§4	Days 1–10	Explain AdminModule wiring; run build/test/lint
S3 file storage & gateway pattern	§5	Days 8–12	Explain PutObject flow + bucket env vars
Docker local stack + Serverless deploy	§6	Days 13–17	Reset Docker stack; list <code>serverless.yml</code> resources
SES, PACR documents, OCR concepts	§7	Days 15–19	Walk invite email path; explain sync vs async OCR
Security (JWT, RLS, secrets)	§8	Days 18–22	Explain Bearer token flow + two secret blast radii
Claude Code (effective use)	§9	Days 20–26	Show read-order + one scoped prompt you used
Critical reasoning (no mind-dump)	§10	Daily + §11	4 AI-free challenges + teach-back on your PR
Ship real code	All	Days 22–30	One merged PR you can explain line-by-line

3.2 Week-by-week sprint

WEEK 1 · DAYS 1–7 — FOUNDATION

Module 4 (NestJS). Docker up, Swagger, read `README.md` + `CONTEXT.md` + `docs/scheduling-workflow.md` . Map project layout. Trace one admin route without AI. Learn scheduling deadline + PACR rules. Daily: 15 min read-before-ask (§10).

WEEK 2 · DAYS 8–14 — DATA & STORAGE

Modules 4–5. Repositories, Drizzle schema (`profiles` , `time_off_requests` , `documents`). Study `s3.gateway.ts` + `serverless.yml` buckets. AI-free challenge week 2. Pick and start first PR with lead.

WEEK 3 · DAYS 15–21 — CLOUD & SECURITY

Modules 6–8. Docker deep-dive (`docs/docker.md`), Lambda/ `lambda.ts` , `npm run deploy` concepts. SES + `docs/onboarding-invite-flow.md` . PACR + OCR concepts (§7). `docs/api-auth.md` , RLS functions, secrets table. AI-free challenge week 3.

WEEK 4 · DAYS 22–30 — AI DISCIPLINE & SIGN-OFF

Modules 9–11 + ship. Claude Code read-order and scoped prompts only. Merge PR with full teach-back. AI-free challenge week 4. Complete every row in §14 sign-off with lead.

3.3 Daily rhythm (repeat all 30 days)

Block	Time	Activity
Morning	30–60 min	Read assigned doc/file for this week’s module (no AI yet)
Midday	2–4 hrs	Hands-on: trace code, exercises, PR work
Afternoon	30 min	Write 3-sentence summary of what you learned (your words)
Before PR	15 min	Teach-back gate (§10.2) — mandatory

4. Module 1 — TypeScript & NestJS on Frontera (days 1–10)

4.1 Project layout

```
src/
  main.ts, lambda.ts      # HTTP + Lambda entry (shared createApp)
  config/                 # TOKENS, AppConfigModule
  admin/                  # Corporate admin modules
  provider/               # Provider portal modules
  scheduling/             # Core domain (grows Q1–Q4)
  repository/
    persistence/          # Drizzle schema + repositories
    aws/                  # S3, SES gateways + AwsModule
  drizzle/                # SQL migrations
  serverless.yml           # AWS deploy
  docker-compose.yml       # Local stack
```

4.2 Patterns we enforce

Pattern	Rule
DI	Inject via <code>TOKENS</code> in <code>src/config/tokens.ts</code>
Controllers	Thin — DTO in, service call, response out
Repositories	All Postgres in <code>repository/persistence/</code>
AWS	All SDK calls in <code>repository/aws/</code> — never <code>new S3Client()</code> in domain services
Config	<code>AppConfigService</code> or constructor <code>readonly</code> fields from <code>ConfigService</code>
Errors	Nest exceptions + <code>src/common/errors/</code> where used

4.3 Week 1 exercises

- ❑ Call `GET /admin/providers` and `GET /health` from Swagger
- ❑ Draw module diagram: `AdminModule` → which services → which repositories
- ❑ Explain scheduling deadline rule (last Tuesday of M–2) in your own words
- ❑ Run `npm run build && npm test && npm run lint`

5. Module 2 — Cloud: file storage (S3) (days 8–12)

Frontera uses **AWS S3** for secure documents (PACR PDFs in Q2+) and export artifacts (Q4). Buckets are provisioned in `serverless.yml`:

- `frontera-{stage}-secure-documents` → `DOCUMENTS_BUCKET`
- `frontera-{stage}-exports` → `EXPORTS_BUCKET`

5.1 Gateway pattern

`S3Gateway` wraps the AWS SDK v3 client. Domain services depend on `IAwsS3Gateway`, not the SDK directly — same pattern you'll use for any future OCR result storage.

```
// Mental model
provider-documents.service
  → S3Gateway.putObject({ bucket, key, body, contentType })
  → documents row in Postgres (metadata + S3 key)
```

5.2 Concepts to master

- Bucket vs key vs region
- IAM permissions on Lambda role (`PutObject` , `GetObject`)
- Pre-signed URLs (future download flows)
- Never expose bucket credentials to the browser — server uploads only

5.3 Exercises

- Read `s3.gateway.ts` and `s3.interface.ts` — list what's missing for `GetObject`
- Trace `provider-documents.service.ts` (if present) or read Q2 notes in `CONTEXT.md`
- Find bucket names in `serverless.yml` and match to `.env.example`
- Write 3 failure modes for an upload (network, IAM, wrong bucket) and expected behaviour

6. Module 3 — Cloud: compute (Docker + Serverless) (days 13–17)

6.1 Docker — local development

```
postgres:16 → migrate (db:migrate + seed) → api (nest start --watch)
              ↑                               |
              └── DATABASE_URL (no SSL) ───┘
```

See `docs/docker.md`. Only `./src` is bind-mounted — rebuild image after `package.json` changes.

6.2 Serverless — production

- Service: `frontera-scheduling`
- Function: `api` → `dist/src/lambda.handler`
- Runtime: Node 22, timeout 300s, HTTP API `{proxy+}`
- Deploy: `npm run deploy` (build + serverless deploy)

6.3 Docker vs Serverless — when to use which

Docker (local / optional host)	Lambda (deployed API)
Fast iteration, hot reload	Production, scales with API Gateway
Full control over process	Pay per request; cold starts possible
Postgres in container	Connects to Supabase pooler in cloud
Good for learning & integration tests	Same Nest code via <code>lambda.ts</code>

Future async work: bulk calendar export or heavy email batches should become enqueue → **202 Accepted** → consumer Lambda (SQS). Don't block the API Lambda on long jobs. `SQS_QUEUE_URL` is reserved in `.env.example` but not wired yet.

6.4 Exercises

- ❑ Run `npm run docker:reset` once — understand migrate + seed
- ❑ Open `src/lambda.ts` and `src/main.ts` — find shared `createApp()`
- ❑ Skim `serverless.yml` — list every AWS resource it creates
- ❑ Explain in writing: what breaks if `FRONTERA_AWS_REGION` doesn't match deploy region

7. Module 4 — Messaging, documents & OCR concepts (days 15–19)

7.1 SES (email)

Provider invites and liaison notifications go through `SesGateway` . Deploy creates SES identity for `SES_FROM_EMAIL` . Read `docs/onboarding-invite-flow.md` .

```
Invite flow (simplified)
Admin creates invite
  → Nest renders HTML email (templates/)
  → SES sends with link to FRONTERA_API_PUBLIC_URL/accept-invite
  → Provider sets password via Supabase Auth
```

7.2 PACR documents (Q2)

Post-deadline schedule changes require a **PACR PDF**. Backend must validate `pacr_document_id` , store file in S3, link `documents` table. This is your **document pipeline** exercise — upload, metadata, authorization.

7.3 OCR — learning path (not in Frontera yet)

Frontera does not call Azure/Texttract today. As a Scalemorph developer you should still understand cloud OCR for other projects:

Concept	Typical pattern	Relates to Frontera when...
Submit document	POST bytes to cloud OCR API	Auto-extract PACR fields (future)
Async poll	202 + Operation-Location → poll until done	Large PDFs can't block HTTP
Store text	Persist in DB or object metadata	Searchable document audit trail
Sync vs queue	Inline in request vs SQS worker	Same trade-off as calendar export

Stretch exercise (offline): read Azure Document Intelligence or AWS Texttract “analyze document” docs and write a 1-page comparison — no code required in Frontera repo.

8. Module 5 — Security (JWT, RLS, secrets) (days 18–22)

8.1 Authentication (Q1 — read now, implement soon)

Supabase Auth issues JWT → Lovable sends `Authorization: Bearer <token>` → Nest guard verifies via `SUPABASE_JWT_SECRET` or JWKS. Full detail: `docs/api-auth.md` .

Routes are **open in local dev** until the guard ships. Always design services assuming `userId` and role will be on the request — don't bake in "no auth forever".

8.2 Row-level security (Postgres)

RLS helpers in `drizzle/0001_supabase_qls_functions.sql`: `has_role`, `get_user_org_ids`, `is_assigned_to`, `log_audit`. Nest uses pooler connection — repository queries must still respect org/site scope when guard is live.

8.3 Secrets — never commit

Variable	Risk if leaked
<code>DATABASE_URL</code>	Full database access
<code>SUPABASE_JWT_SECRET</code>	Forged tokens
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Bypass all RLS
<code>AWS_ACCESS_KEY_ID</code> / profile	S3/SES abuse, deploy rights

9. Module 6 — Working with Claude Code (days 20–26)

Claude Code / Cursor agents are part of how Scalemorph works — on **Fontera**, point them at the right docs first.

9.1 Read order before asking Claude to code

1. `CONTEXT.md` (Frontera repo)
2. `docs/scheduling-workflow.md`
3. Relevant `docs/adr/000N-*.md` for your feature
4. `CLAUDE.md` — stack conventions (note: some lines may still reference other projects; trust `CONTEXT.md` for Frontera)

9.2 Good vs poor use

Good	Poor
"Explain <code>schedule-change-approvals.service.ts</code> flow" after you read it	"Build Q3 review queue" with no ADR context
Generate DTO + test scaffold; you fill business rules	Ship generated scheduling deadline logic unchecked
"Add PutObject to S3Gateway matching existing style"	"Add AWS" with new patterns that bypass gateway

9.3 Frontera-specific prompt examples

```
"Read docs/adr/0006-schedule-change-approvals.md and list
which repository methods the service calls."

"In s3.gateway.ts add getObject returning Buffer — match putObject
style, no new dependencies."

"Review my diff for secrets in logs and missing work_site_id scoping."
```

10. Module 7 — Critical reasoning (don't mind-dump on AI) (daily, all 30 days)

AI makes you fast; it does not make you correct. On Frontera, wrong scheduling rules or leaked PACR paths have real compliance implications — you must understand what you ship.

10.1 Daily habits (15–30 min)

Habit	Action
Read before ask	3-sentence summary of a file before Claude explains it
Predict before run	Write expected API response; compare to Swagger/curl
Explain aloud	Weekly: explain invite flow or deadline rule without notes
Bug journal	Expected / actual / hypothesis / fix — you write it

10.2 Frameworks

5 Whys (example)

1. Why was time-off rejected? → Missing PACR
2. Why required? → Past scheduling deadline
3. Who enforces deadline? → Nest service (not frontend alone)
4. What date logic? → Last Tuesday of M–2
5. Where is helper? → Find it in codebase before changing

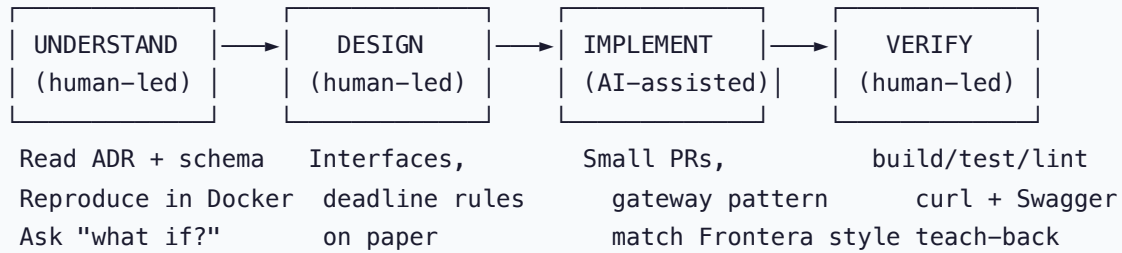
Teach-back gate (before every PR)

- What user role can call this endpoint?
- Which tables change?
- What happens on validation failure?
- How did you test without AI writing the test for you?

10.3 AI-free weekly rotation (4 weeks)

Week	Challenge
1	Trace <code>GET /admin/onboarding/form-options</code> with grep only
2	Fix a type error without autocomplete; explain deadline rule aloud
3	Read <code>serverless.yml</code> and draw AWS resources on paper
4	Teach-back on your PR diff with zero AI — every line explained

11. Understand → Design → Implement → Verify



Rule: AI may help in *Implement* only if you fully own the other three stages.

12. Week-by-week checklist — complete everything in 30 days

Week 1 — Foundation · Module 4 (days 1–7)

- ❑ `npm run docker:up` — health + Swagger working
- ❑ Read `README.md`, `CONTEXT.md`, `docs/scheduling-workflow.md`
- ❑ Explain scheduling deadline + PACR rule in your own words
- ❑ Trace one admin route end-to-end — AI-free (challenge week 1)
- ❑ Run `npm run build && npm test && npm run lint`
- ❑ 1:1 with lead — confirm full 30-day curriculum plan

Week 2 — Data & storage · Modules 4–5 (days 8–14)

- ❑ Map `src/admin/` + `src/provider/` → services → repositories
- ❑ Study `schema.ts` for `profiles`, `time_off_requests`, `documents`
- ❑ Complete Module 5 exercises (`s3.gateway.ts`, buckets in `serverless.yml`)
- ❑ AI-free challenge week 2 (type fix + deadline explain aloud)
- ❑ First PR opened with lead-approved scope

Week 3 — Cloud & security · Modules 6–8 (days 15–21)

- ❑ Complete Module 6 exercises (Docker reset, `lambda.ts`, serverless resource list)
- ❑ Complete Module 7 (SES invite flow + OCR sync vs async write-up)
- ❑ Complete Module 8 (`api-auth.md`, RLS helpers, four secrets table)
- ❑ AI-free challenge week 3 (draw AWS resources from `serverless.yml`)
- ❑ PR in review — every changed line explainable

Week 4 — AI discipline & sign-off · Modules 9–11 (days 22–30)

- Apply Module 9 Claude Code workflow on your PR (read-order + scoped prompts)
- Daily critical-reasoning habit maintained (Module 10)
- AI-free challenge week 4 (teach-back PR with zero AI)
- PR merged
- Every row in §14 30-day sign-off ticked with lead

13. Reading list & repo map

Read in this order

1. `README.md`
2. `CONTEXT.md`
3. `docs/scheduling-workflow.md`
4. `docs/docker.md`
5. `docs/api-auth.md`
6. `docs/onboarding-invite-flow.md`
7. `docs/adr/0002-scheduling-workflow-backend-shape.md`
8. `.env.example`
9. `serverless.yml`
10. `src/repository/persistence/db/schema.ts`
11. `src/repository/aws/`

External references

- NestJS — modules, providers, guards
- Drizzle ORM — schema, migrations
- AWS S3 SDK v3 — PutObject, GetObject
- AWS SES — SendEmail, domain verification
- Serverless Framework — Lambda + HTTP API
- Docker Compose — multi-service local stacks

14. 30-day sign-off — full curriculum

All items required by day 30. This is the bar for “grasped everything” in this guide.

Area	Demonstration	Done
Frontera domain	Explain scheduling deadline + PACR rule without notes	☐
NestJS	Small feature using TOKENS + repository pattern	☐
Docker	Reset stack, explain postgres → migrate → api	☐
S3	Explain gateway pattern and bucket env vars	☐
Serverless	Describe what <code>npm run deploy</code> provisions	☐
SES	Walk through provider invite email path	☐
Security	Explain JWT flow + name two secrets and their blast radius	☐
OCR (concepts)	Explain sync vs async document processing trade-offs	☐
AI discipline	PR where you explain every changed line	☐
Reasoning	One AI-free weekly challenge + one teach-back	☐

Developer _____	Date _____
Lead (Hamza) _____	Sign-off date _____